



Lecture 41

Wrapping Things Up

CS61B, Fall 2025 @ UC Berkeley

61B in 12 Minutes or Less

Lecture 41, CS61B, Fall 2025

61B in 12 Minutes or Less

Your Future

Course Reflections

AMA

So Long!

Where We Started

Just 14 weeks ago:

```
Sublime Text  File  Edit  Selection  Find  View  Goto  Tools  Project  Window  Help
hello.py
1 print("hello world")

hello.py
[Finished in 31ms]

HelloWorld.java
1 public class HelloWorld {
2     public static void main(String[] args) {
3         System.out.println("hello world");
4     }
5 }

HelloWorld.java
[Finished in 386ms]

Person), 2025-(
Line 37, Column 213
Line 69, Column 10
Line 3, Column 43, Saved - /Dropbox/FA25/lec2/helloWorld.java (UTF-8)
Tab Size: 4  Python
Tab Size: 4  Java
```

[61B FA25] Lecture 1 - Intro

What We've Learned about Programming Languages

- Object oriented programming:
 - Interface Inheritance. 
 - Implementation inheritance.
 - Generic Programming, e.g. ArrayList<Integer>, etc.
 - The model of memory as boxes containing bits. 
 - Bit representation of positive integers (hashing lecture).
 - Java.
 - Some standard programming idioms/patterns:
 - Objects as function containers (e.g. Comparators).
 - Default method specification in interfaces.
 - Iterators.
- Example: Programmer only needs to know List API, doesn't have to know that ArrayList secretly does array resizing.**
- Example: Array is a sequence of boxes. An array variable is a box containing address of sequences of boxes.**

Important data structure interfaces:

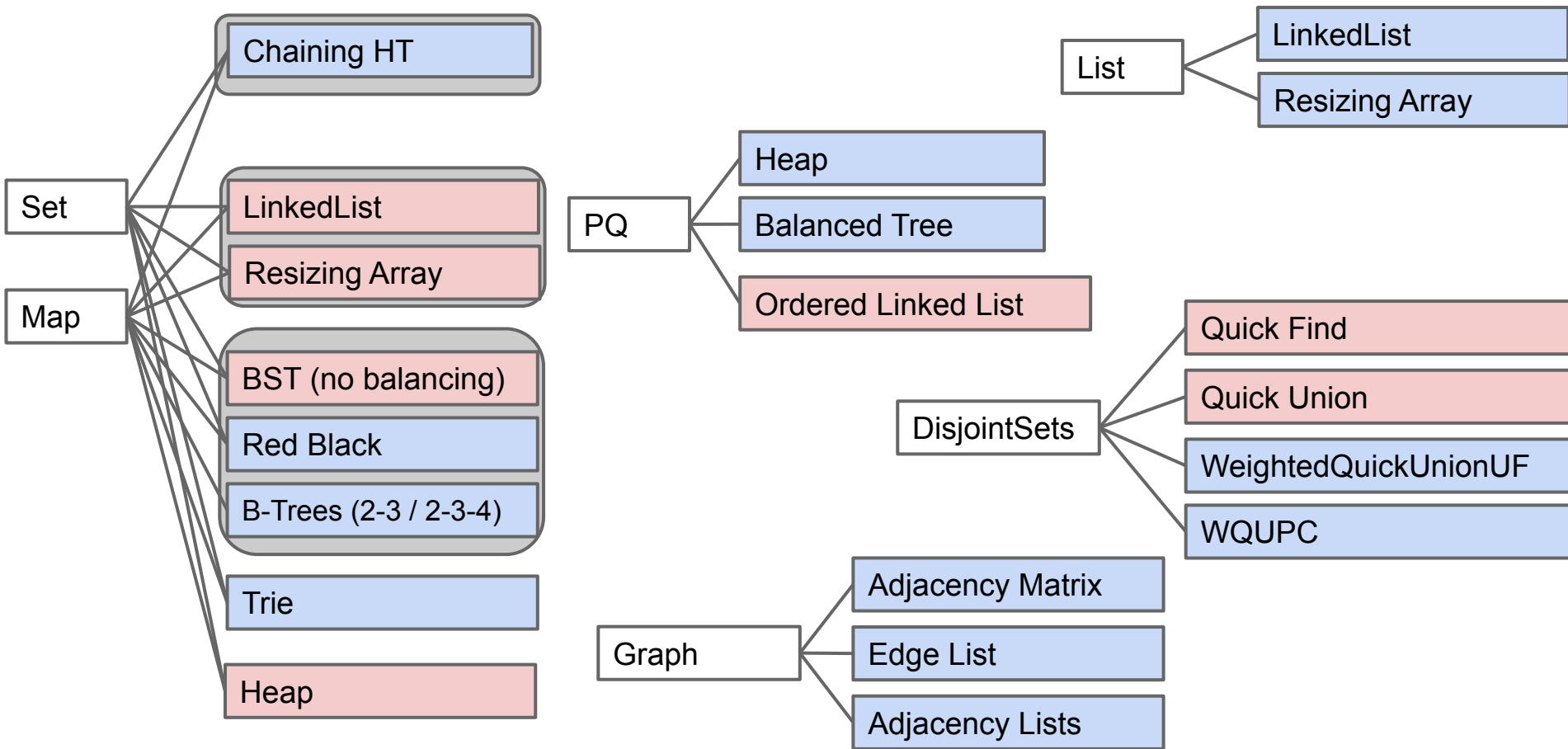
- `java.util.List`
- `java.util.Set`
- `java.util.Map`
- Our own Collections (e.g. `Map61B`, `Deque61B`)

Concrete implementations of these abstract implementations:

- Examples: `ArrayDeque61B` implements `Deque61B`.

- Asymptotic analysis.
- $O(\cdot)$, $\Omega(\cdot)$, and $\Theta(\cdot)$.
- Worst case vs. average case vs. best case.
 - Exemplar of usefulness of average case: Quicksort
- Determining the runtime of code through inspection (often requires deep thought).
- Multiplicative resizing is much better than additive resizing.
 - Multiplicative resizing results in $\Theta(1)$ add runtime for ArrayLists.
 - Additive resizing results in $\Theta(N)$ add runtime for ArrayLists.

Some Examples of Implementations for ADTs



Array-Based Data Structures:

- ArrayLists and ArrayDeque
- HashSets, HashMaps, MyHashMap: Arrays of 'buckets'
- ArrayHeap (tree represented as an array)

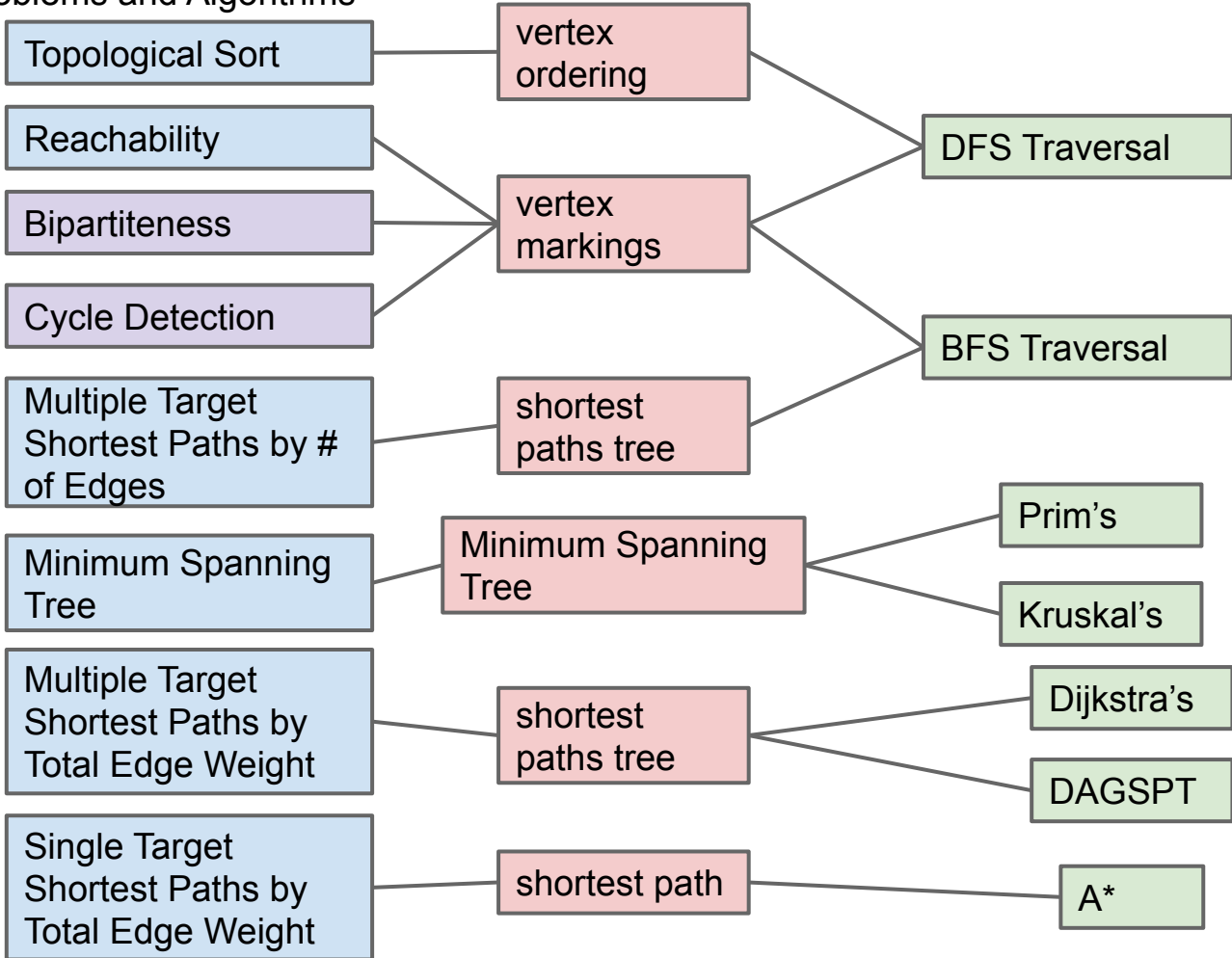
Linked Data Structures

- Linked Lists
 - LinkedList, IntList, LinkedListDeque, SLList, DLList
- Trees: Hierarchical generalization of a linked list. Aim for bushiness.
 - TreeSet, TreeMap, BSTMap, Tries (trie links often stored as arrays)
- Graphs: Generalization of a tree (including many algorithms).

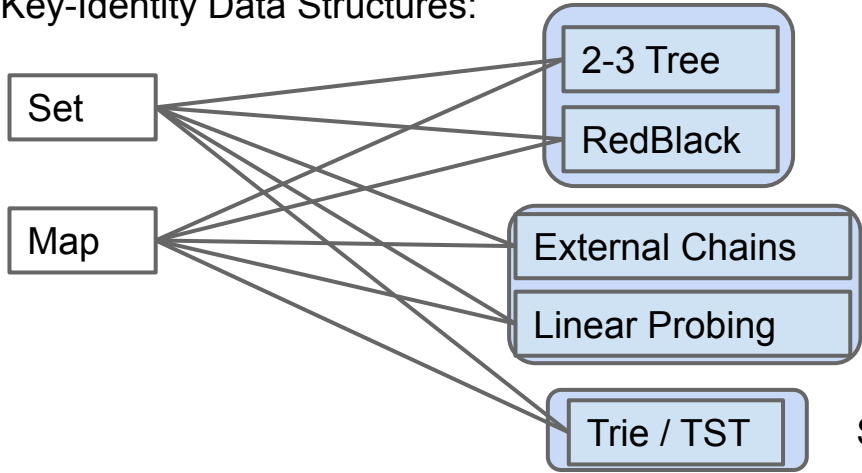
Tradeoffs of array-based vs. linked data structures.

Tractable Graph Problems and Algorithms

Discussed
in section



Search-By-Key-Identity Data Structures:

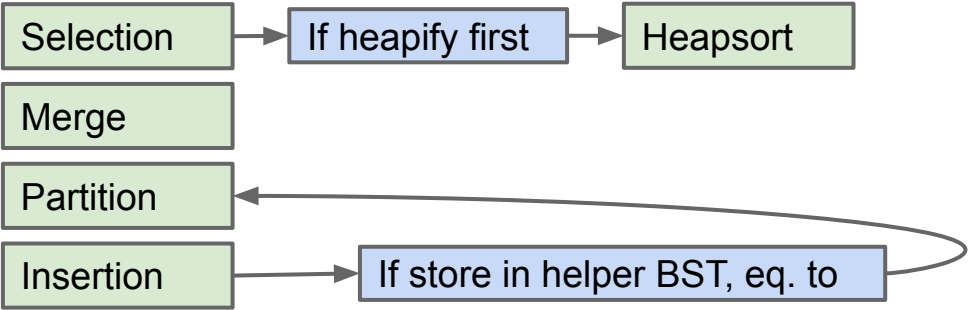


Searches using compareTo()
Analogous to **Comparison-Based**

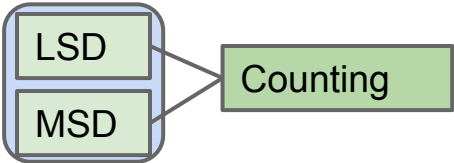
Searches using hashCode() and equals()
Roughly Analogous to **Integer Sorting**

Searches by digit. Analogous to **Radix Sorting**

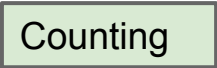
Comparison Based Sorting Algorithms: $\Omega(N \log N)$ worst case



Radix Sorting Algorithms: $\Theta(NW)$ worst case:
(require a sorting subroutine)



Small-Alphabet (Integer) Sorting Algorithms: $\Theta(N)$ worst case



- Java syntax and idioms.
- Unit testing (and its more extreme form: Test-driven development).
- Asking the internet and LLMs for help.
- Debugging:
 - Identify the simplest case affected by the bug.
 - Hunt it down, giving it no place to hide.
 - With the right methodology, can find bugs even when finding bug through manual code inspection is impossible.
- Tactical vs. Strategic Programming.
 - Avoid information leakage. Minimize dependencies. Minimize obscurity.
- Real tools: IntelliJ, git, Truth, command line.
- Data structure selection (and API Design)
 - Drive the performance and implementation of your entire program.

Your Future

Lecture 41, CS61B, Fall 2025

61B in 12 Minutes or Less

Your Future

Course Reflections

AMA

So Long!

Two interesting questions:

- What can you do?
- What should you do?

Some Josh Hug History

[illegible]

Quoting Justin Yokota: “I would argue that the most important thing your grade tells you is how hard your next semester will be.

Most of the time, you don't truly master a concept until you're forced to use it to solve something new. A lot of the concepts we've covered in the last half of the course will only solidify when you start taking 61C, 170, or any other "next step" class.

The grade you get in this course is my message to you saying how difficult I predict it will be to develop that mastery in your next class.”

My strong belief: With sufficient high quality deliberate practice, you can all get **very good** at any topic you study here at Berkeley.

You are a rare commodity.

Revenue

Meta	\$164.5 billion
Alphabet (Google)	\$350.0 billion
Amazon	\$638.0 billion
Microsoft	\$245.0 billion
Apple	\$391.0 billion

Sources: [Link](#), [Link](#)

Revenue per Employee

Meta	\$2,221,000
Alphabet (Google)	\$1,909,000
Amazon	\$410,000
Microsoft	\$1,075,000
Apple	\$2,384,000

The skills you are building will be in high demand from companies, non-profits, government agencies, educational institutions, and more.

- The choice of how to spend your career is yours.

Quite a lot of you will likely end up working at some sort of technology company at some point in your life.

There's nothing (IMO) wrong with working at profit driven tech companies.

- Please do realize that even as a rank and file employee, you have the power to effect change, particularly if you are paid in stock (because then you are a partial owner).
 - Example: Sectarian violence in Myanmar driven by Facebook posts was a huge problem due to a lack of local language speakers. Nobody stepped up to fix this.
 - Example: Frances Haugen (former Facebook data scientist) leaked documents showing shady behavior by the company.

And of course realize that your skills can be used in many ways!

My request: Use your superpowers in a way that is a net positive for the lives of your fellow humans.

- I'm not saying that working for big companies is evil or anything, or that you have a moral obligation to work for relatively low wages for the public good (though that'd be great, of course).
- but keep in mind that your work will profoundly affect the world.

(Wanna talk about this stuff more? Take CS195!)

Course Reflections

Lecture 41, CS61B, Fall 2025

61B in 12 Minutes or Less
Your Future

Course Reflections

AMA

So Long!

CS61B Version 4.0 (Fall 2025)

- Many more smaller assignments, especially at the beginning.
 - No more project 0.
 - Not sure if this was successful. Maybe too tedious?
- Flexible points policy for HWs.
 - Previously we had a flexible deadlines policy for HWs.
- Gave vague permission for students to use LLMs on HWs.
- Midterm 1 as a checkpoint for knowing how to write code from scratch.
- Opt-in Attendance (ha).
- Focused course material even further on what matters (less syntax, less inheritance, less Java trivia).
- Renamed all the labs to just be HWs.

Main changes I have in mind:

- Not teaching CS70 at the same time.
- Get students back in to rooms with each other (discussion attendance was 10% this semester, lecture was similar).
 - Target: 50% discussion attendance (which is what 61A is doing)
- Tentatively: More open-ended programming challenges for students with especially strong background.
- Tentatively: Pare back the swarm of early assignments.
 - Shift slightly back towards “flashy” assignments rather than “directly related to lecture” assignments.
- Tentatively: Midterm 1 is on a computer and retakable.

We read all of your feedback, especially on the end of course evaluations.

- There is an official form from the university, and an internal survey for 61B that we'll send out ourselves.

AMA

Lecture 41, CS61B, Fall 2025

61B in 12 Minutes or Less

Your Future

Course Reflections

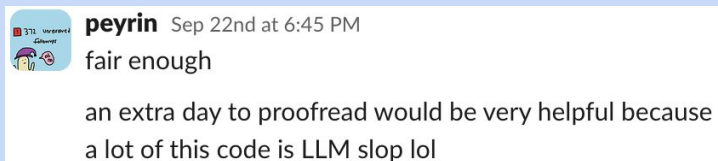
AMA

So Long!

Questions About Anything* for Josh and Peyrin?

What happened to Gitlet and BearMaps (old design projects)?

Do you use LLMs to create assignments?



How do I learn how to build cool things?

Can you discuss your cybercrimes?

How do you feel about the NeetCode 150 roadmap?

The software engineering market is volatile. How disposable are we?

Differences between engineering/industry job and academia job?

Has the market ever been this volatile since you started teaching?

Are post-GPT classes dumber than pre-GPT classes?

What is the successor to 61B? 61C feels more complementary than a sequel.

Favorite data structure?

What do you love about teaching?

Your most famous alumni?

Do people still create new data structures?

What is your best public speaking tip?

What's a class you haven't taught, but want to?

How do you keep up-to-date with learning new things about CS? (<https://www.quantamagazine.org/>)

If you teach 61B forever, will you teach other classes too?

For someone with a passion in activism, how do you not let your ideals go?

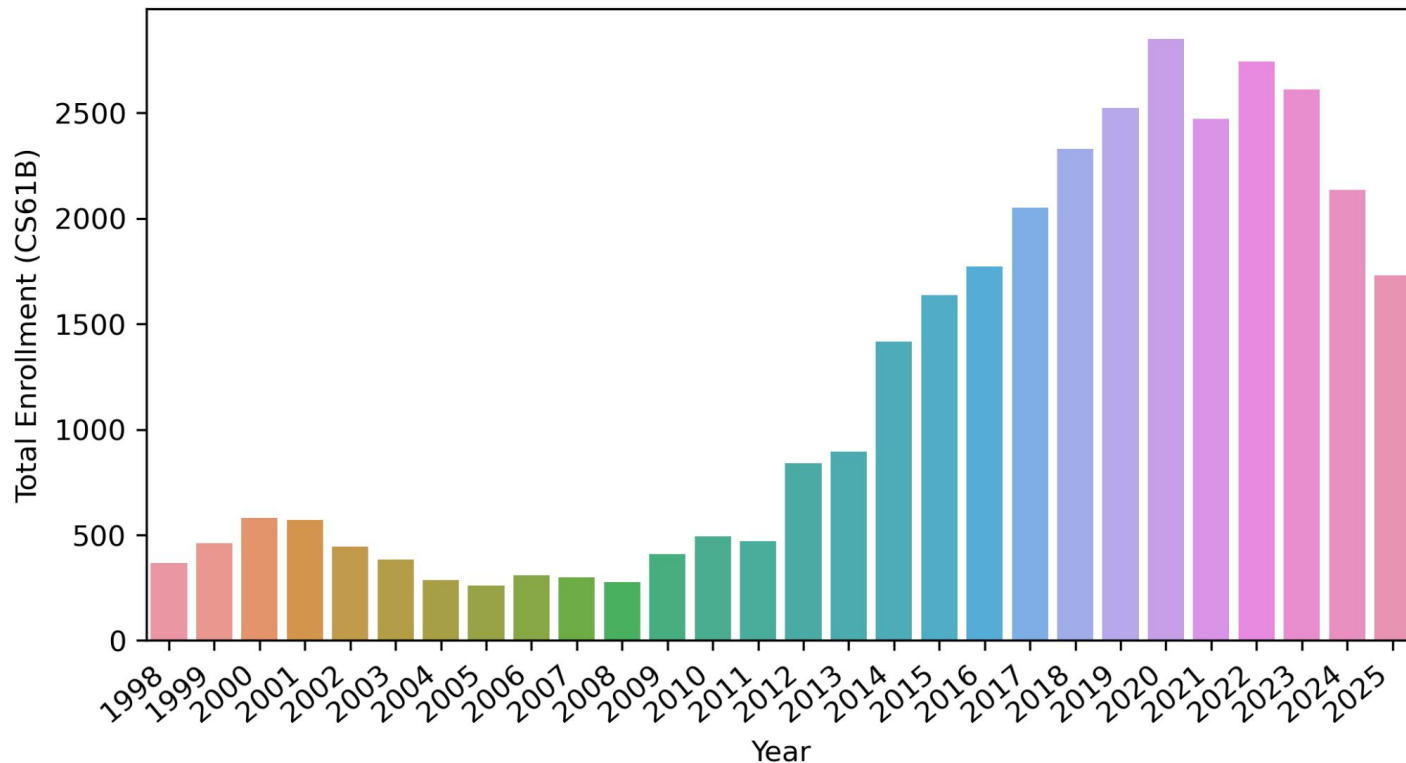
So Long!

Lecture 41, CS61B, Fall 2025

61B in 12 Minutes or Less
Your Future
Course Reflections
AMA
So Long!

61B Would Love to Have You Next Spring

- Teaching interns: Learn more, help others, get units.
- Watch out for an announcement on the Fall 2025 61B Ed.



Special Thanks to the Staff (who keep the wheels on the bus)

Head TAs: Daniel Wang, Elaine Shu, Stacey Lei

TAs: Aditya Hariharan, Amy Wang, Anirudh Kotamraju, Dawn Schumacher, Iris Zhou, Karen Meng, Natalia Grondin, Ricardo Sandoval, Ritika Joshi, Ronnie Beggs, Ryan Yang, Tiffany Li, Yinqi Yi

Tutors: Andrew Schoenen, Avik Gautam, Ayush Gupta, Codey Ma, Dennis Koh, Isabel Dong, Jonathan Gasser-Brennan, Lawrence Wu, Lilah Durney, Marcus Koh, Michelle Koga, Miller Liu, Phoenix Wilson, Rushil Saraf, Sarveshan Saravanan

Teaching Interns (TIs): Alex Nava, Anay Shukla, Brendan Reyes, Deven Mittal, Elaine Liu, Emily Huynh, Ethan Yang, Ishanth Hombaiah, John Hu, Jonathan Cao, Krishna Chunduri, Kuljit Uppal, Monishka Tanwani, Saksham Jain, Sruthi Vegunta, Theo Sohn, Zhen Liu

Closing Thoughts (Josh)

It is a terrifying and awesome responsibility to decide how you should spend hundreds of your precious hours here at Berkeley.

I hope I've done a good job. I know it wasn't perfect (and sadly never will be).

- (but I've got Ideas™)

... and thanks for all the kind words, feedback, and understanding when things are not as good as they could be.